

C++ and overloaded operators

BIE-PA2 – Programming and Algorithmic II

Ladislav Vagner

Department of Theoretical Computer Science
Faculty of Information Technology
Czech Technical University in Prague
`xvagner@fit.cvut.cz`

March 2, 2026

Overview

- Motivation for operator overloading – rational numbers.
- `friend` functions.
- Encapsulation
- Conversion constructors.
- Overloaded operators – an overview.
- Compilation units and `make`.

Rational numbers – class CRat

- Member variables – integer numbers representing numerator and denominator.
- Methods – clear from the declaration below.
- The arithmetic operations are defined outside the class.

```
class CRat {  
    int num_, den_; // numerator, denominator  
    void simplify(); // simplify the fraction  
public:  
    CRat(int num = 0, int den = 1);  
    int numerator() const { return num_; }  
    int denominator() const { return den_; }  
    void print(std::ostream& out = std::cout) const;  
};  
CRat add(CRat, CRat);  
CRat neg(CRat);  
CRat sub(CRat, CRat);  
bool equal(CRat, CRat);  
...
```

Rational numbers – class CRat

constructor – initialize, positive denominator, simplification:

`CRat ( ),`

simplification – if numerator is zero then denominator = 1,
both numerator and denominator are divided by the
greatest common divisor, private: `simplify ( ),`

print – display numerator / denominator in the output
stream: `print ( ),`

addition – find common denominator (least common
multiplier), add, simplify: `add ( ),`

subtraction – find common denominator (least common
multiplier), subtract, simplify: `sub ( ),`

comparison – compare numerators and denominators: `equal ( ),`

...

Note: Our fractions are always simplified and denominator is a positive number.

Rational numbers – class CRat

```
...  
CRat::CRat(int num, int den) : num_(num), den_(den) {  
    if (den == 0)  
        throw std::invalid_argument("denominator is zero");  
    simplify();  
}  
  
void CRat::print(std::ostream& out) const {  
    out << '(' << num_ << '/' << den_ << ')';  
}  
  
void CRat::simplify() {  
    if (den_ < 0) {  
        num_ = -num_;  
        den_ = -den_;  
    }  
    int x = std::gcd(num_, den_);  
    num_ /= x; den_ /= x;  
}  
...
```

Rational numbers – class CRat

```
...  
CRat add(CRat x, CRat y) {  
    return {  
        x.numerator() * y.denominator()  
        + y.numerator() * x.denominator(),  
        x.denominator() * y.denominator()  
    };  
}  
  
CRat neg(CRat x) {  
    return { -x.numerator(), x.denominator() };  
}  
  
CRat sub(CRat x, CRat y) {  
    return add(x, neg(y));  
}  
  
bool equal(CRat x, CRat y) {  
    return x.numerator() == y.numerator() &&  
        x.denominator() == y.denominator();  
}  
...
```

Rational numbers – class CRat

Sample use:

```
...  
int main() {  
    using namespace std;  
  
    CRat a(3, 5), b(1), c;  
  
    a.print(); cout << ' '; b.print();  
    cout << endl;  
  
    c = add(a, b);  
    c.print(); cout << endl;  
  
    c = sub(a, b);  
    c.print(); cout << endl;  
  
    cout << equal(sub(b, a), a) << endl; // (b - a) == a  
  
    return 0;  
}
```

Rational numbers – class CRat

- The solution is correct (from the function point of view).
- The solution is wrong from the design point of view:
 - add, sub, and equal functions must use getters to access the member fields.
 - It would be logical to include the methods into the class definition,
- The use of print is clumsy:
 - The stream operation must be canceled,
 - print function is called and
 - then the stream output may continue.

Rational numbers – class CRat II.

- The operations (add, sub, and equal) became methods of the class.
- The operations have only one parameter (the first parameter is the instance itself). Former first parameter (the left operand) is now the instance itself.

```
class CRat {
    int num_, den_;
    void simplify();
public:
    CRat(int num = 0, int den = 1);
    // int numerator() const;
    // int denominator() const;
    void print(std::ostream& out = std::cout) const;
    CRat add(CRat) const;
    CRat neg() const;
    CRat sub(CRat) const;
    bool equal(CRat) const;
};
...
```

Rational numbers – class CRat II.

Sample use. The left-hand operand of the add, sub, and equal operations is the instance on which the method is invoked.

```
...  
int main() {  
    using namespace std;  
  
    CRat a(3, 5), b(1), c;  
  
    a.print(); cout << ' '; b.print();  
    cout << endl;  
  
    c = a.add(b);  
    c.print(); cout << endl;  
  
    c = a.sub(b);  
    c.print(); cout << endl;  
  
    cout << b.sub(a).equal(a) << endl; // (b - a) == a  
    return 0;  
}
```

Rational numbers – class CRat III.

It would be both convenient and easy-to-understand if the program is written like:

```
...  
int main() {  
    using namespace std;  
  
    CRat a(3, 5), b(1), c;  
  
    a.print(); cout << ' '; b.print();  
    cout << endl;  
  
    c = a + b;  
    c.print(); cout << endl;  
  
    c = a - b;  
    c.print(); cout << endl;  
  
    cout << (b - a == b) << endl;  
  
    return 0;  
}
```

Overloaded operators may be used to introduce this syntax.

Rational numbers – class CRat III.

```
class CRat {
    int num_, den_;
    void simplify();
public:
    CRat(int num = 0, int den = 1);
    int numerator() const { return num_; }
    int denominator() const { return den_; }
    void print(std::ostream& out = std::cout) const;
};

// we must use getters again
CRat operator + (CRat, CRat);
CRat operator - (CRat);
CRat operator - (CRat x, CRat y) { return x + -y; }
bool operator == (CRat x, CRat y) {
    return x.numerator() == y.numerator() &&
           x.denominator() == y.denominator();
}
...
```

Rational numbers – class CRat III.

- The overloading of an operator gives a new meaning to an existing operator for new data types.
- The overloaded operator is nothing but a short name for a function or a method.
- Syntax and properties of operations (priority and associativity of the operator) is unchanged.
- The compiler searches for the overloaded operators implemented as functions or methods.
- For CRat, the overloaded operators are only functions:

```
c = a + b;    // compiled as: c = operator + ( a, b );  
c = a - b;    // compiled as: c = operator - ( a, b );  
x = a == b;   // compiled as: x = operator == ( a, b );
```

Rational numbers – class CRat IV.

- The overloaded operators were functions in the previous example.
- Encapsulation issue disappears if methods are used.
- Use of them is unchanged, but the compiler calls method `CRat::operator + (CRat) const` instead of `operator + (CRat, CRat)`.

```
class CRat {  
    int num_, den_;  
    void simplify();  
public:  
    CRat(int num = 0, int den = 1);  
    // int numerator() const;  
    // int denominator() const;  
    void print(std::ostream& out = std::cout) const;  
    CRat operator + (CRat) const;  
    CRat operator - () const;  
    CRat operator - (CRat y) const { return *this + -y; }  
    bool operator == (CRat) const;  
};
```

Friend functions

- Functions do not have access to `private` and `protected` fields in class T.
- The same applies to methods in classes not related to the class T (inherited classes are an exception).
- Selected functions (and classes) may be granted the right to access `private` or `protected` fields in a class.
- The access is granted by the `friend` keyword.
- The access must be granted inside class declaration.
- Friend functions and classes violate encapsulation (up to some level). The technique shall be used rarely, in justified cases. The friend functions de-facto became a part of the class definition:
 - the structure of the class is exposed to the friend functions,
 - a change to the class structure must be reflected in the friend functions.
- Friend functions are useful for operator overloading, namely for `operator <<`.

Intermezzo – why encapsulation

A lot of programmers believes that encapsulation is OOP:

- there are `private` member variables in a class,
- constructor sets member variables to its parameters,
- there are `public` getters for all members,
- there are `public` setters for all members,
- yes, this is the true OOP.

Intermezzo – why encapsulation

A lot of programmers believes that encapsulation is OOP:

- there are **private** member variables in a class,
- constructor sets member variables to its parameters,
- there are **public** getters for all members,
- there are **public** setters for all members,
- yes, this is the true OOP.

The above is a complete misunderstanding of encapsulation and OOP principles. Encapsulation is a mean, it is not the goal. The goal is a clear, enforceable and high-level public interface. Thus we can:

- modify class implementation without breaking the rest of the program (e.g. use more efficient algorithm or different data representation),
- completely replace the class, the new class just needs to match the public interface,
- fix classes without risk to the rest of the program,
- develop classes simultaneously (team of programmers).

Intermezzo – why encapsulation

What is "clear and enforceable public interface"?

- Public interface: can be used by the rest of the program, shall be as stable as possible,
- non-public (private) interface: used internally by the class, may be changed if needed,
- compiler uses encapsulation rules to check that private interface is not used where not allowed.

What is "high-level public interface"?

- Public interface shall follow the intended operation (e.g. add two rational numbers),
- public interface shall be straightforward, i.e. shall not be bloated with unnecessary details (e.g. how to find least common multiplier, simplification),
- the interface shall allow all required operations, **but not more**,
- if the interface is high-level, it usually does not have to expose implementation details. Thus further improvements/bug fixes are less likely to break the rest of the program.

Intermezzo – why encapsulation

Getters and setters comply with encapsulation. However, reckless use of getters and setters denies OOP principles:

- private member with public getter and setter is de-facto public (as if declared public):
 - there is something the setter can do, like validate the value,
 - setter may start some further computation (e.g. modify other member variables),
 - however public setter makes it very difficult to modify anything in the internal data representation.
- If public getters and setters exist for all (most) member variables, struct may be used instead of class,
- if getter/setter is included in public interface, the class is tied to some implementation which is difficult to change.

Intermezzo – why encapsulation

Friend functions add exceptions to the encapsulation rules.

However, reasonably designed friend functions may comply with OOP principles:

- `friend` declaration indeed spans private interface to the given function,
- `friend` functions are usually functions that should have been implemented as methods, however, there were some reasons not to include them as methods (e.g. the type/order of arguments),
- the class and its `friend` functions are usually implemented in the same module. Thus a modification to the class structure requires updates just to this one module and not to the rest of the program,
- `friend` declarations shall be used with extreme caution.

Rational numbers – class CRat V.

The functions are ordinary functions (thus require two arguments), however, opposed to ordinary functions, these have direct access to **private** and **protected** member variables directly.

```
class CRat {
    int num_, den_;
    void simplify();
public:
    CRat(int num = 0, int den = 1);
    friend std::ostream& operator << (std::ostream&, CRat);

    friend CRat operator + (CRat, CRat);
    friend CRat operator - (CRat x) {
        return { -x.num_, x.den_ };
    }
    friend CRat operator - (CRat x, CRat y) {
        return x + -y;
    }

    friend bool operator == (CRat x, CRat y) = default;
};
```

Rational numbers – class CRat V.

The implementation allows even more:

```
int main() {  
    using namespace std;  
  
    CRat a(3, 5), b(1);  
    int i = 1;  
  
    b = 2;          // CRat = int !!  
    cout << b << endl;  
    b = a + i;      // CRat + int !!  
    cout << b << endl;  
    b = 2 - a;      // int - CRat !!  
    cout << b << endl;  
  
    return 0;  
}
```

Rational numbers – class CRat V.

The implementation allows even more:

```
int main() {  
    using namespace std;  
  
    CRat a(3, 5), b(1);  
    int i = 1;  
  
    b = 2;          // CRat = int !!  
    cout << b << endl;  
    b = a + i;      // CRat + int !!  
    cout << b << endl;  
    b = 2 - a;      // int - CRat !!  
    cout << b << endl;  
  
    return 0;  
}
```

Even more surprising is the fact the results are correct!

User conversions

- Type Foo may be implicitly converted into type Bar if either:
 - type (class) Bar has a constructor which may be called with one argument of type Foo (user defined conversion constructor), or
 - type Foo has an overloaded cast operator to type Bar
- The conversion constructor creates a temporary object which is used for the operation (i.e. = / add / sub on the previous slide).
- The constructor:

```
CRat ( int num = 0, int den = 1);
```

is a user-defined conversion constructor, the conversion is `int` to `CRat`.

- The operations are thus performed:

```
b = 2;          // b = CRat ( 2 );  
b = a + 2;      // b = a + CRat ( 2 );  
b = 2 + a;      // b = CRat ( 2 ) + a;
```

User conversions

- If operator – was a method of CRat, the code 2 – a would not compile, because the conversion is never applied to the object a method is called on.
- Cast operator is for instance:

```
struct CRat {  
    ...  
    operator double() const { // note: no return type  
        return double(num_) / den_;  
    }  
};
```

- The automatically performed conversion may be suppressed if keyword `explicit` is used in constructor declaration:

```
struct CRat {  
    ...  
    explicit CRat( int num = 0, int den = 1 );  
    // always use explicit with bool  
    explicit operator bool() const;  
};
```

User conversions

- The existence of both the user conversion constructor in Foo and cast operator to Foo (or very similar type) leads to issues:

```
struct CRat {  
    CRat(int num = 0, int den = 1);  
    operator double() const;  
    friend CRat operator + (CRat, CRat);  
};  
  
int main() {  
    CRat a(3, 5);  
    auto b = a + 1; // ambiguous overload  
}
```

- The situation can be solved by marking (at least) one of those as `explicit`, but it may be even better not to have these at all.

Overloaded operators – an overview

- There are certain limitations in operator overloading:
 - only existing operators may be overloaded. It is not possible to introduce a new operator (e.g. operator \$),
 - the arity or precedence of operators cannot be changed (e.g. operator += must be binary),
 - overloading may add a new semantic, it cannot replace existing semantic. For example, operator + may be overloaded for our class CRat (C++ does not have any built-in semantic for it), however, it would be illegal to overload the operator for int data type (C++ already knows how to add integers).
- Do not recklessly overload operators:
 - overloaded operators make sense in math (e.g. our rational numbers, big numbers, vectors, ...) and strings (concatenation),
 - collections (like vectors, sets, maps, ...) may use overloaded operators,
 - other classes may require overloaded operators <<, >>, and =,
 - when in doubt, prefer method over overloaded operator.

Overloaded operators – overview

- Can't be overloaded: `::`, `.`, `.*`, `?:`
- Can be overloaded as a method only: `=`, `()`, `[]`, `->`
- Unary: `&`, `*`, `!`, `~`, `+`, `-`
- Common infix: `+`, `-`, `*`, `/`, `%`, `^`, `&`, `|`, `<<`, `>>`
- Combined infix: `+=`, `-=`, `*=`, `/=`, `%=`, `^=`, `&=`, `|=`, `<<=`, `>>=`
- Increment / decrement: `++`, `--`
- Comparison: `==`, `!=`, `<`, `>`, `<=`, `>=`, `<=>` (spaceship operator)
- Logical binary operators: `&&`, `||`
- Cast: `operator` Foo
- Special: `new`, `new[]`, `delete`, `delete[]`, `co_await`, `""_suffix`, `->*`, `,` (comma)
- If the operator is implemented as a method, the object on which the operator is called never participates in conversion.

Overloaded operators – input and output

- Operators `>>` and `<<`, if overloaded to work with streams, need to be overloaded as functions.
- Usually they are `friend` functions.
- Left argument is always `std::istream&` or `std::ostream&`, not a specialized type like `std::istreamstream&`.
- The return type is the same as the left argument, to allow call chaining.
- Typical header thus are:

```
std::ostream& operator << (std::ostream&, const Foo&);  
std::istream& operator >> (std::istream&, Foo&);
```

Overloaded operators – input and output

Example:

```
std::ostream& operator << (std::ostream& out, CRat x) {  
    return out << '(' << x.num_ << '/' << x.den_ << ')';  
}  
  
std::istream& operator >> (std::istream& in, CRat& x) {  
    char l, s, r;  
    int n, d;  
    if (!(in >> l >> n >> s >> d >> r)  
        || l != '(' || s != '/' || r != ')'  
        || d == 0) { // Avoid CRat constructor exception  
        // Mark the stream as failed  
        in.setstate(std::ios::failbit);  
    } else {  
        x = { n, d };  
    }  
  
    return in;  
}
```

Overloaded operators – common infix

- Common infix operators:

- +, -, *, /, %, ^, &, |, <<, >>
- Overloaded typically as free functions.
- Can be implemented using combined operator variant:

```
Foo operator @ (Foo a, const Foo& b) {  
    a @= b; return a; }
```

- Combined operators @=:

- +=, -=, *=, /=, %=, ^=, &=, |=, <<=, >>=
- Formally independent on their common variants.
- It is strongly recommended to maintain the expected semantics, such that `x @= y` is the same as `x = x @ y`.
- Typically overloaded as methods.
- Can be implemented using the common operator variant:

```
Foo& Foo::operator @= (const Foo& b) {  
    return *this = *this @ b; }
```

- Both delegations can't be used together.

Overloaded operators – increment / decrement

- Typically overloaded as methods.
- Prefix variant:

```
Foo& Foo::operator ++ (); // return reference  
Foo& operator ++ (Foo&);
```

- Can often be implemented using addition / subtraction:

```
Foo& Foo::operator -- () { return *this -= 1; }
```

- Postfix variant:

```
Foo Foo::operator ++ (int); // return value  
Foo operator ++ (Foo&, int);
```

- Common implementation:

```
Foo Foo::operator -- (int) {  
    Foo tmp = *this;  
    operator -- (); // prefix --  
    return tmp;  
}
```

- The int parameter is always 0, unless called as function `operator++(2)`.

Overloaded operators – comparison

- Since C++20 the compiler tries:
 - change the order of operands with `==` and `!=`,
 - convert `x != y` to `!(x == y)`, and
 - transform inequalities to `<=>` (new spaceship operator).
- It is also possible to generate the implementations of `<=>` and `==`:
 - Attributes are compared one by one in order of their declaration in the class.
 - Generated `<=>` triggers automatic generation of `==`.
 - User defined `<=>` does not trigger generation of `==`.
 - The syntax:

```
struct Foo {  
    friend auto operator <=> (const Foo&, const Foo&)  
        = default;  
    friend bool operator == (const Foo&, const Foo&)  
        = default;  
};
```

- Before, all 6 former operators had to be implemented by hand. Though delegation and `std::tie` simplified the process.

Overloaded operators – `<=>`

- Spaceship (aka three-way comparison) operator `<=>`.
- Expected implementation needs `#include <compare>`.
- The return type denotes the ordering class:
 - `std::strong_ordering`
 - `std::weak_ordering` – equal values may be distinguishable,
 - `std::partial_ordering`
- All three contain constants:
 - `less`, `greater`, `equivalent`,
 - `equal` (only `strong_ordering`), and
 - `unordered` (only `partial_ordering`).
- Implementation of `==` using `<=>` is not recommended due to performance reasons.
- It holds that $(a \text{ <=> } b) < 0$, iff $a < b$. Analogically for `>`, `<=`, `>=`, `==`, and `!=`.

Overloaded operators – indexation

- Can only be implemented as a method.
- Typically overloaded twice:

```
struct Foo {  
    Bar& operator [] (size_t index);  
    const Bar& operator [] (size_t index) const;  
};
```

- The argument can be of any type, not just `size_t`.
- Since C++23, there can be more arguments (but we only use C++20).
- Implementation can be delegated, but it requires `const_cast`, helper template function, or deducing-this (since C++23).

Overloaded operators – user literals

- User suffixes must start with _.
- Integer literals:

```
Foo operator ""_foo (unsigned long long);  
Foo operator ""_foo (const char*);
```

- Floating point literals:

```
Foo operator ""_foo (long double);  
Foo operator ""_foo (const char*);
```

- String literals:

```
Foo operator ""_foo (const char*, size_t);
```

- Character literals:

```
Foo operator ""_foo (char);
```

- Must be declared or defined outside of a class, ADL is not used.

Overloaded operators – user literals

- Typical example: `"foo"s` is a literal of `std::string` type.
- To use it, use `using namespace std::literals`.
- Example:

```
CRat operator ""_r(unsigned long long x) {  
    return CRat(x);  
}  
  
int main() {  
    auto x = 1_r / 3; // CRat  
    std::cout << x << std::endl; // (1/3)  
}
```

Overloaded operators – pointers / iterators

- Overloading `->` and unary `*` makes an object behave like a pointer.
- Operator `->` must return a pointer or an object with `->` operator.
- For iterators we need `++`, `--`, `+`, `==`, ...
- Typically implemented as methods.

```
struct FooPtr {  
    FooPtr(Foo* ptr) : ptr_(ptr) {}  
    Foo*      operator -> ()      { return ptr_; }  
    const Foo* operator -> () const { return ptr_; }  
    Foo&      operator * ()       { return *operator->(); }  
    const Foo& operator * () const { return *operator->(); }  
private:  
    Foo *ptr_;  
};  
  
Foo foo;  
FooPtr p(&foo);  
p->bar();
```

Overloaded operators – leftovers

- Function call `()`: Object can be used as a function. TBD as part of the lecture on STL.
- Assignment `=`: TBD as part of the lecture on copying.
- Unary `&` does not typically make sense to overload.
- Unary `!` as well, it is better to overload `operator bool`.
- Logical operators `&&` and `||` lose their short-circuit evaluation when overloaded.
- Before C++17 operators `&&`, `||` and `,` lost their sequencing property (i.e. evaluation of the left argument before the right one).
- Unary `-` can be used to implement binary `-`, but the performance may not be optimal:

```
Foo operator - (const Foo& a, const Foo& b) {  
    return a + -b;  
}
```

Translation units

Also known as modules (but do not confuse with C++20 modules).

- Big projects are usually divided into many source files:
 - header files `.h` with the declarations of classes and functions,
 - module files `.cpp` with the implementation of functions and methods.
- Advantages:
 - source files are short and easy to understand,
 - individual modules may be developed by different programmers,
 - universal modules may be shared among many projects (e.g., as libraries),
 - modules may speed up project compilation.
- Disadvantages:
 - difficult to compile manually,
 - specialized tools are required for efficient compilation (`make`, `qmake`, IDE support).

Compilation – a revision

- Preprocessor processes individual `.c/.cpp` modules:
 - header files are read and their contents is copied into the output,
 - constants and macros are expanded,
 - conditional compile removes unwanted source code.
- Compiler reads the output of the preprocessor:
 - the syntax is checked,
 - the source is converted into a memory representation,
 - the representation is transformed and optimized (e.g. common subexpressions, loop inlining, register allocation, ...),
 - machine code is generated,
 - the result is stored as an object file (`.o/.obj`).
- Linker connects all generated object files:
 - standard and additional libraries are added,
 - imports and exports are paired,
 - the executable file is generated.

Translation units

- Manual compilation is simple if the program consists of a single module:
 - `g++ -Wall -pedantic -o exename file.cpp`
 - `g++` executes preprocessor, compiler, and linker, in this order,
 - the result is the executable,
 - the intermediate files are erased (e.g., `.o` files).
- Manual compilation with many modules:
 - the compiler is executed for each module (`.c/.cpp` source file):
`g++ -Wall -pedantic -c mod.cpp`
 - the linker is executed once, with all generated `.o/.obj` files:
`g++ -Wall -pedantic -o exename mod1.o mod2.o ...`

Translation units

- The speed of compilation may be increased:
 - existing object files are not deleted,
 - the compiler is executed only for the modified `.c/.cpp` files,
 - if a header changes, all `.c/.cpp` modules using the header are re-compiled,
 - the linker is always executed.
- Why this helps:
 - the compiler is typically slow (optimizations),
 - we usually modify only few modules out of the entire project,
 - the modules may be compiled in parallel (using several CPUs),
 - the linker is relatively fast (it searches identifiers and fills-in tables).

Translation units

Example:

- our program consists of three modules: `mod1.cpp`, `mod2.cpp`, and `main.cpp`,
- modules `mod1.cpp` and `mod2.cpp` do have their corresponding header files `mod1.h` and `mod2.h`,
- module `main.cpp` uses functions from the other modules, i.e., it includes their header files.

A complete compilation:

```
> g++ -Wall -pedantic -c mod1.cpp
> g++ -Wall -pedantic -c mod2.cpp
> g++ -Wall -pedantic -c main.cpp
> g++ -Wall -pedantic -o acme_program main.o mod1.o mod2.o
```

Translation units

Example:

- our program consists of three modules: `mod1.cpp`, `mod2.cpp`, and `main.cpp`,
- modules `mod1.cpp` and `mod2.cpp` do have their corresponding header files `mod1.h` and `mod2.h`,
- module `main.cpp` uses functions from the other modules, i.e., it includes their header files.

A change to `mod1.cpp`:

```
> g++ -Wall -pedantic -c mod1.cpp  
> g++ -Wall -pedantic -o acme_program main.o mod1.o mod2.o
```

Translation units

Example:

- our program consists of three modules: `mod1.cpp`, `mod2.cpp`, and `main.cpp`,
- modules `mod1.cpp` and `mod2.cpp` do have their corresponding header files `mod1.h` and `mod2.h`,
- module `main.cpp` uses functions from the other modules, i.e., it includes their header files.

A change to `mod1.h`:

```
> g++ -Wall -pedantic -c mod1.cpp
> g++ -Wall -pedantic -c main.cpp
> g++ -Wall -pedantic -o acme_program main.o mod1.o mod2.o
```

Translation units – make and Makefile

- Manual compilation is laborious:
 - the commands are long and error-prone,
 - it is boring to manually search for the modified .cpp files,
 - it is even more boring to manually search for the modified header files and then compute the transitive closure of all affected modules.
- A simple script (e.g. a shell script) does not help (modified header files).
- `make` is a program designed to handle compilation steps with explicit dependencies,
- `Makefile` is a text file with the list of compilation steps and a list of dependencies:
 - program `make` reads `Makefile`,
 - it searches for the missing and modified files,
 - it executes the commands to re-generate the results that depend on the missing or modified files (e.g., it executes compilation, linking, ...).

Translation units – make and Makefile

- A Makefile typically lists some variable definition and a set of rules to generate targets.
- A rule is a list of commands that convert the prerequisites into the target file.
- A target is a file, that may be generated automatically from its prerequisites. An example target is the desired executable file, or some intermediate file (e.g., `.o/.obj`, ...).
- A prerequisite is a source file, or a different target file (e.g., `.c/.cpp`, `.h`, or `.o/obj`).
- Program `make` searches the first rule and its target. It is customary to name the first target `all`.
- The rules are then searched recursively for each prerequisite.
- The recursion stops once there is not rule to generate a prerequisite. Such prerequisite is a source file provided by the programmer (`.c/.cpp/.h`).

Translation units – make and Makefile

- Each target visited by the recursion may be re-generated. A target will be generated and its rules will be executed if:
 - the target does not exist,
 - or it exists, but its modification time is before the modification time of some of its prerequisites. That is, the prerequisite has been modified since the target was recently generated.
- Makefile usually lists target `clean`, the target contains rules to erase all intermediate files:
 - when forcing a clean build,

```
> make clean
```

```
> make
```
 - when sources are published.
- For historical reasons, each command in a rule must start with the TAB character.

Translation units – Makefile

```
# variables, e.g. to easily switch between gcc and clang
CXX=g++
LD=g++
CFLAGS=-Wall -pedantic

all: acme_program

acme_program: main.o mod1.o mod2.o
    $(LD) -o acme_program main.o mod1.o mod2.o

main.o: main.cpp mod1.h mod2.h
    $(CXX) $(CFLAGS) -c main.cpp
mod1.o: mod1.cpp mod1.h
    $(CXX) $(CFLAGS) -c mod1.cpp
mod2.o: mod2.cpp mod2.h
    $(CXX) $(CFLAGS) -c mod2.cpp

clean:
    rm -f acme_program main.o mod1.o mod2.o
```

Translation units – make and Makefile

Usage:

- Compile and link all missing/outdated targets:
 > make
- Cleanup:
 > make clean
- Build a specific target (with all its dependencies):
 > make main.o
- Try to parallelize, make spawns up to 4 parallel processes:
 > make -j 4

Translation units – Makefile

- Developing a Makefile is not much fun. However, it may be partly automated:
 - make sets several variables: target name ($\$@$), the name of the first prerequisite ($\$<$), the list of all prerequisites ($\$^$), and many others. With the use of variables, the compilation rule may read:

```
main.o: main.cpp mod1.h mod2.h
        $(CXX) $(CFLAGS) -c $<
```

- make understands macros, the macros are applied based on the target and the first prerequisite names (extensions). For example, the following macro is applied to all .o targets built from .cpp modules:

```
%.o: %.cpp
        $(CXX) $(CFLAGS) -c $<
```

- the list of modules and their prerequisites may be generated automatically:

```
> g++ -MM *.cpp >> Makefile
```

Translation units – Makefile

```
# variables, e.g. to easily switch between gcc and clang
CXX=g++
LD=g++
CFLAGS=-Wall -pedantic
OBJS=main.o mod1.o mod2.o

all: acme_program

acme_program: $(OBJS)
    $(LD) -o $@ $^

clean:
    rm -f acme_program $(OBJS)

%.o: %.cpp
    $(CXX) $(CFLAGS) -c $<

# automatically generated by g++ -MM

main.o: main.cpp mod1.h mod2.h
mod1.o: mod1.cpp mod1.h
mod2.o: mod2.cpp mod2.h
```

Questions ...